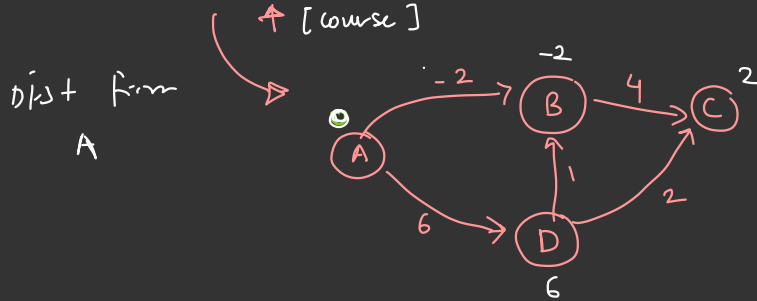


"Graphs - 4"

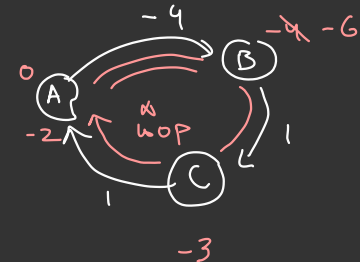
Udemy Course
(upto DFS & Backedges)

→ Shortest Paths

- BFS - Single Source Shortest Path on unweighted graphs
- greedy → Dijkstra's Algo - Single source shortest Path on weighted graphs - dir + undir
- Bellman Algo - SSSP on graphs with -ve edges but no -ve wt cycle



SSSP is not possible



DP • Floyd Warshall $O(N^3)$

gives dist
b/w every
pair of nodes



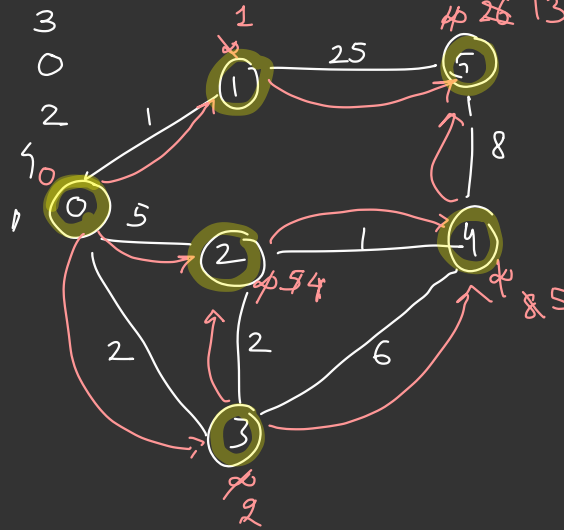
Delhi - Mumbai
Goa - Amritsar

N cities
↳
 $N C_2$ pairs

◦ Dijkstra's Algo (Single source shortest path)

Path from 0-5

src	dest	<u>cost</u>	<u>parent</u>
0-1	1	1	0
0-2	4	3	3
0-3	2	0	0
0-4	5	2	2
0-5	13	5	4
0-0	0	0	0



5-4-2-3
1
1

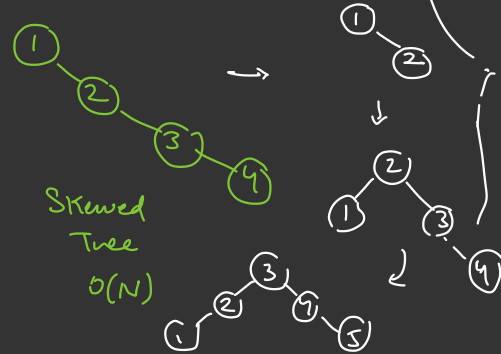
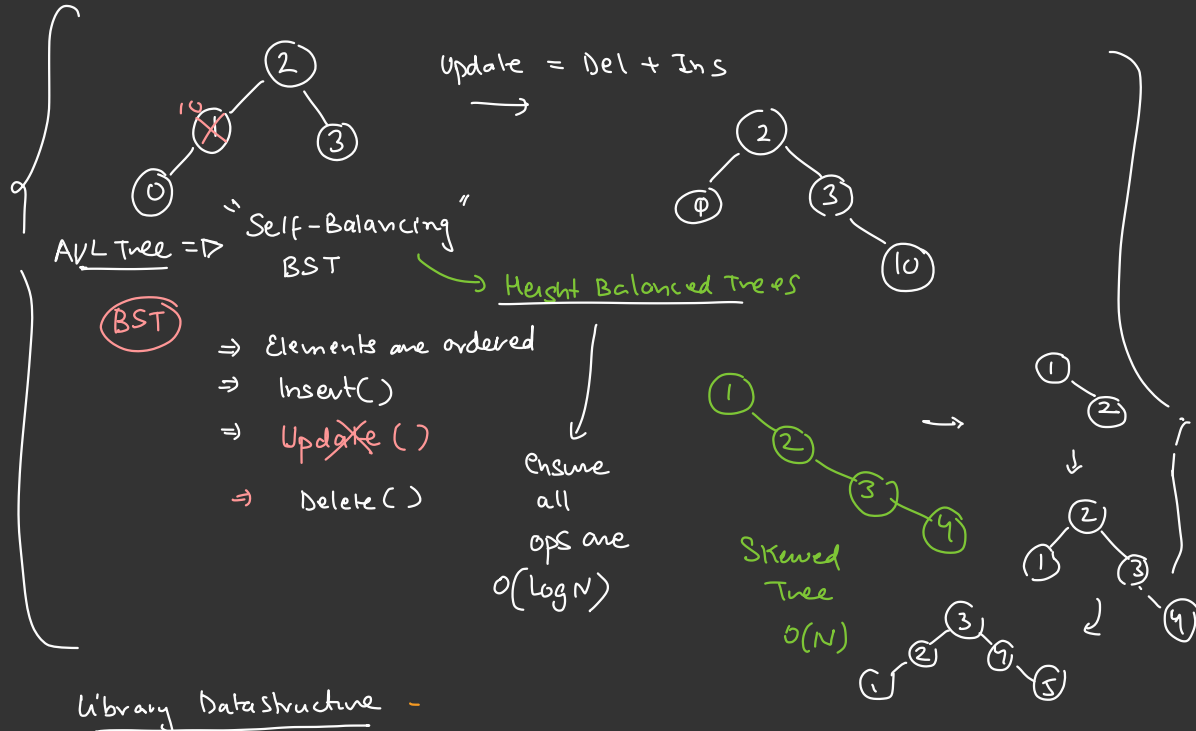
No direct of updating in PQ
PQ - add (-)
- getMin()
- removeMin()

Dist	0	4	2	5	13	
Node	0	1	3	2	4	5

↑ ↑ ↑ ↑

~~Priority Queue~~ based upon distance

TreeSet ordered acc to dist
↑



```
class Pair {
    int node
    int dist
}
```

3

C++ set()
~~unordered-set()~~

Java
TreeSet()
~~HashSet()~~

Python
ordered-set()

Dijkstra's (src, adjList[])

$\Rightarrow \underline{\text{dist}}[] = \text{init}() \quad [\infty, \infty, \dots]$

$\text{dist}[\text{src}] = 0$

$\text{Set} < \text{pair} < \overset{d}{\downarrow} \text{int}, \overset{n}{\downarrow} \text{int} > > \quad S = \text{TreeSet}()$

$S.\text{insert}(\text{pair}(0, \text{src}))$

init

You might need
a comparator

(a, b)

(c, d)

while (!S.empty()) {

Pair p = S.pollFirst()

n = p.node

d = p.dist

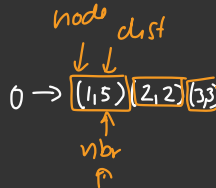
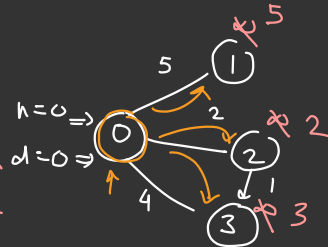
for (nbr : adjList[n]) {

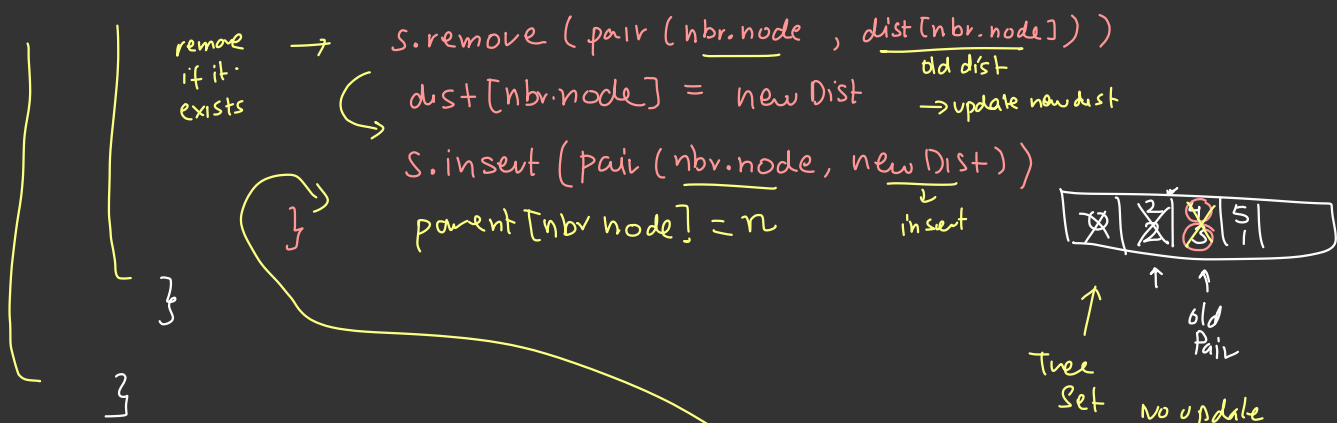
newDist = d + nbr.dist

if (newDist < dist[nbr.node]) {



pollFirst()





```

for (i=0; i < N, i++) {
    print (Node → i
           dist → dist[i])
}

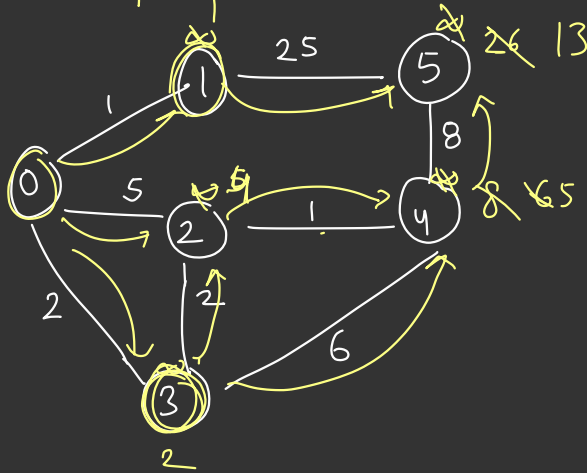
```

// print the path as well by maintaining a parent array

}

dist $\rightarrow$

0	1	2	3	4	5	6
--------------	--------------	--------------	--------------	--------------	--------------	--------------



$0 \rightarrow (\underline{1}, 1) (\underline{2}, 5) (\underline{3}, 2)$
 $1 \rightarrow (\underline{0}, 1) (\underline{5}, 25)$
 $2 \rightarrow (\underline{0}, 5) (\underline{3}, 2) (\underline{4}, 1)$
 $3 \rightarrow (\underline{0}, 2) (\underline{2}, 2) (\underline{4}, 6)$
 $4 \rightarrow (\underline{2}, 1) (\underline{5}, 8) (\underline{3}, 6)$
 $\underline{5} \rightarrow (\underline{1}, 25) (\underline{4}, 8)$

while (!S.empty())

$n, d = S.\text{poll}(\text{first}())$

for (every nbr of n) {

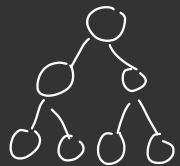
update the dist if nd is less than prev.

Remove + Insert in set $\Rightarrow \log$

}

E Edges

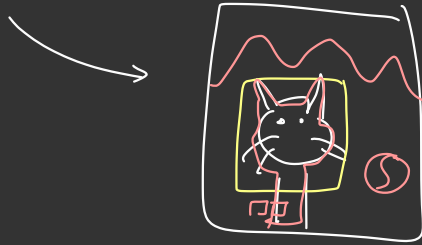
V nodes/
vertices



Max Set

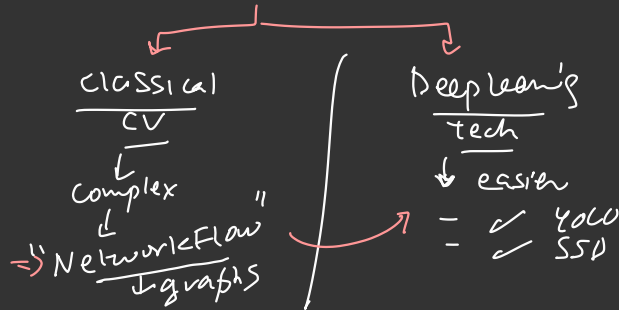
Size V

Short
code



Computer Vision

- Classify
- locate (Box)
- Segmentation



Each Ins/→ $\text{Log } V$
Del

$\boxed{\text{TC}} \rightarrow \boxed{O(V + E \log V)}$

greedy Algo



ordered Tree set



greedy choice



node with
min dist
first

Disjoint Set Union [DSU]

(Udemy Course - 1 hr)

↑

DSU dsu(n)

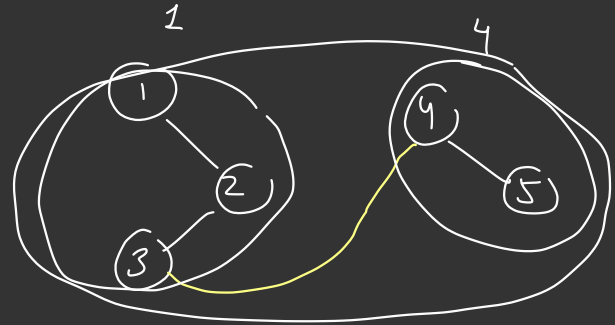
→ Find(i) → set id it belongs to

→ Join(i, j) → joins the sets in
or union(i, j) which i and j
lie

join(3, 4)

10.45

n = 5



Smaller id as the
id of
new set

DSU $\text{dsu}(5)$

$\text{find}(2) \rightarrow 2$

$\text{find}(3) \rightarrow 3$

$\text{union}(0, 2)$

$\text{find}(2) \rightarrow 0$

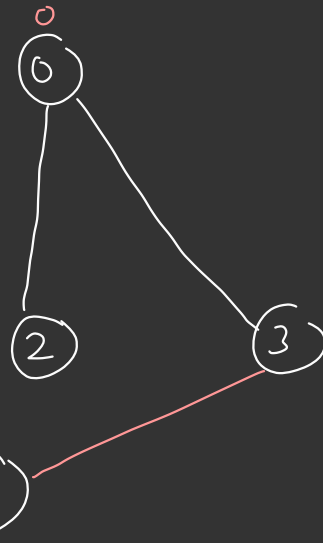
$\text{union}(3, 4)$

$\text{union}(0, 3)$

$\text{find}(3) \rightarrow 0$

$\text{find}(4) \rightarrow 0$

¹
①



Concept

$\Rightarrow$ Do ② & ③ belong to same set?

$\downarrow$ $\downarrow$
 $\text{find}(2)$ $\text{find}(3)$
 $\downarrow$ $\downarrow$
 $0 == 0$

$\Rightarrow$ Do ① & ④ belong to same set?

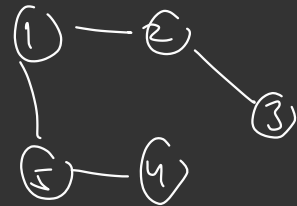
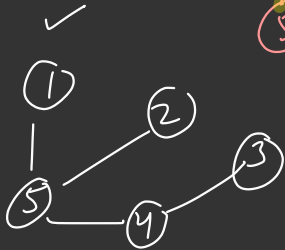
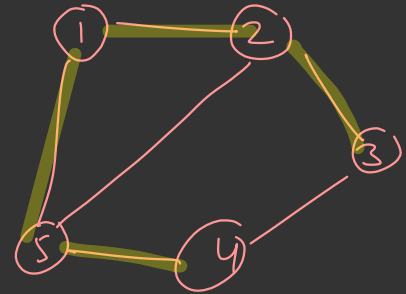
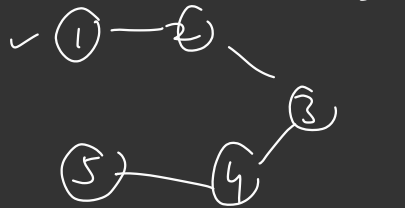
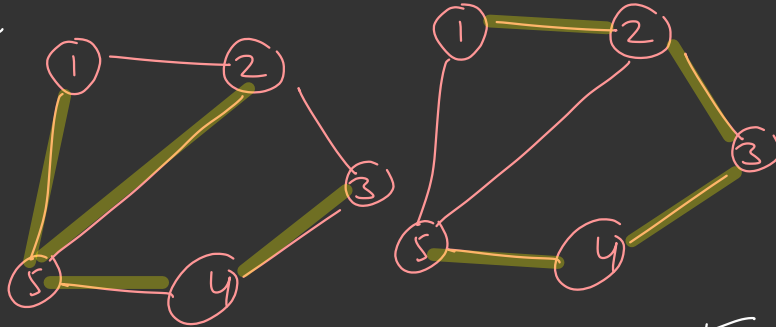
Spanning Tree (of a graph)

subset of edges that connects all nodes of a graph
in a tree like fashion

N Nodes $\rightarrow$ $N-1$ edges st all nodes are
picked connected

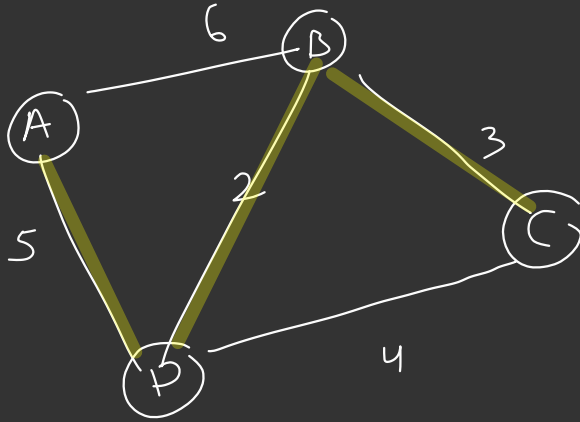
No
cycle

in
Spanning tree



Minimum Spanning Tree (weighted edges)

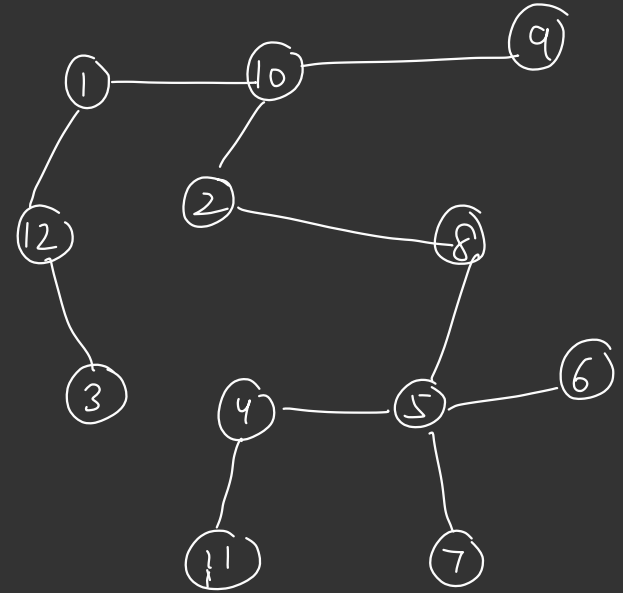
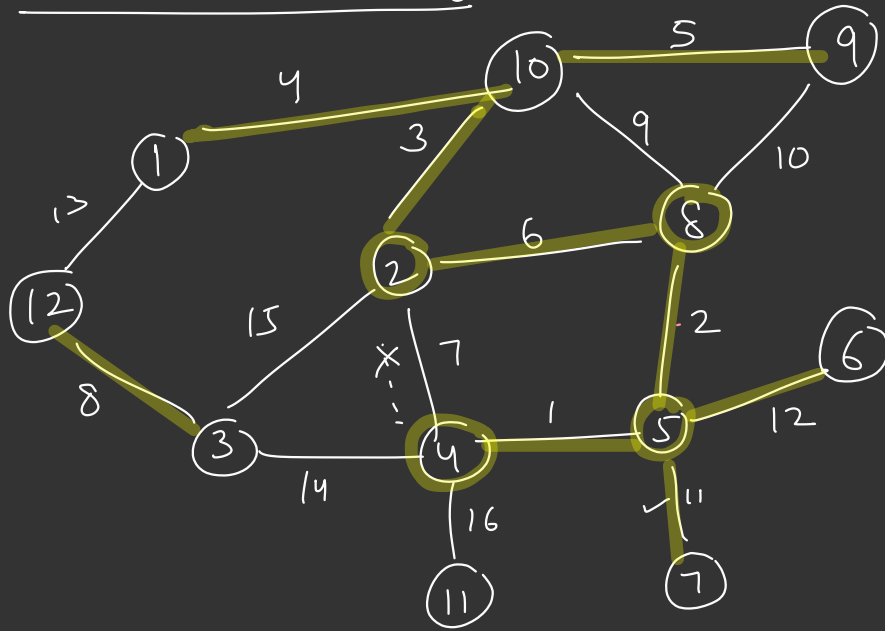
a subset of edges that creates a spanning tree of
min wt of edges
total



$$5 + 2 + 3 = \underline{\underline{10}}$$

↑
Min Cost

Kruskal's MST Algo



Step-1 Sort the edges by wt
 $\{ 1, 2, 3, 4, 5, \dots, 16 \}$

$$O(E \log E + E \log V)$$

"edgelist"

Algo

edges = sort(all edges acc to wt)

= $[(x_1, y_1, wt_1) (x_2, y_2, wt_2) \dots]$

DSU dsu(n)

ans = 0

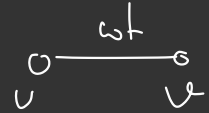
for(edge : edges){

u → edge.u

v → edge.v

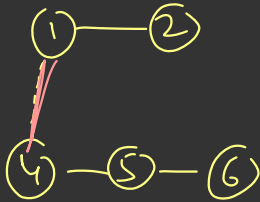
wt → edge.wt

cycle → if (dsu.find(u) != dsu.find(v)) {
parent dsu.union(u, v)
 ans = ans + wt
 }



class edge {
 int u
 int v
 int wt
 }

edge edges[] =



}

→ log V